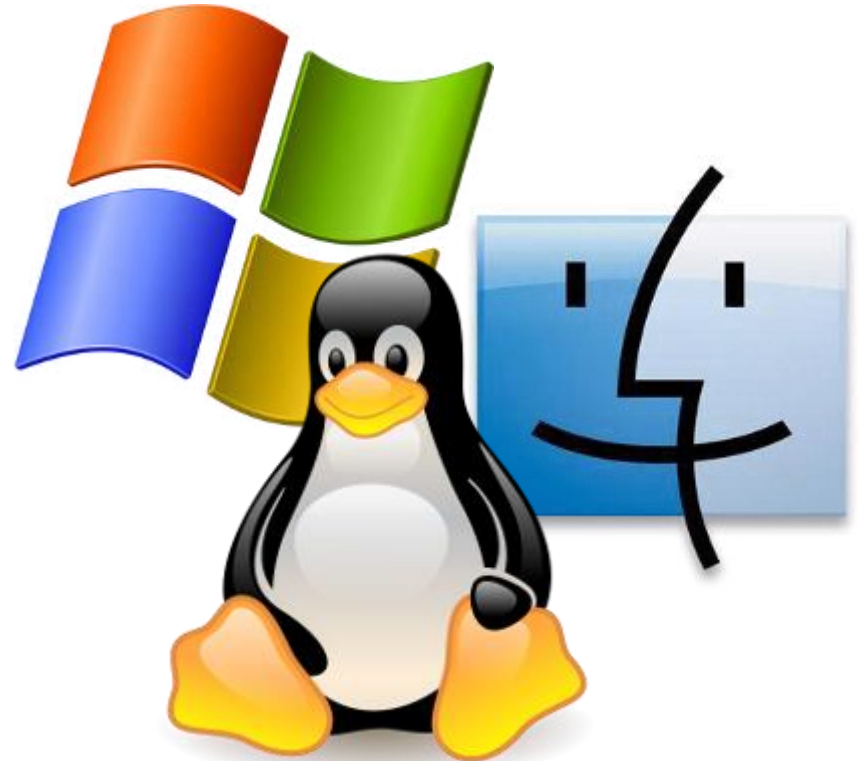


2 – Prozessverwaltung

Tobias Werner M. Sc.
Uni Bayreuth – Angewandte Informatik 3
Leibniz-FH Hannover

`tobias.werner.doz@leibniz-fh.de`

(in Teilen nach der Vorlesung
"Betriebssysteme" von Prof. Dominik Henrich
an der Universität Bayreuth)



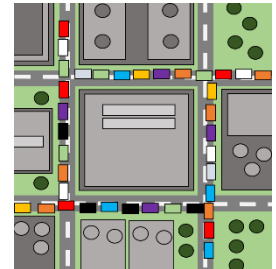
[Logos von Windows 7, Linux und Mac OS X]

Einleitung

Heranführung

- Angenommen, man bestellt drei Heimwerker, um eine Wohnung zu renovieren.
Die Heimwerker arbeiten zuverlässig, aber stur und ohne Nachdenken ihre Aufgaben ab.
Jeder Heimwerker braucht einen vollen Tag für seine Arbeit.

→ Ich bestelle für Montag den Elektriker, für Dienstag den Tapezierer
und für Mittwoch den Maler. Was passiert?
→ Ich bestelle alle drei für "Anfang der Woche". Was passiert?
- Warum gibt es im folgenden Bild einen Stau und was macht man dagegen?



Kompetenzen

- Prozesse und Threads als Konzepte kennen und unterscheiden lernen
- Hintergründe der Rechenzeituteilung verstehen
- typische Probleme der Paralleldatenverarbeitung erkennen und beheben lernen

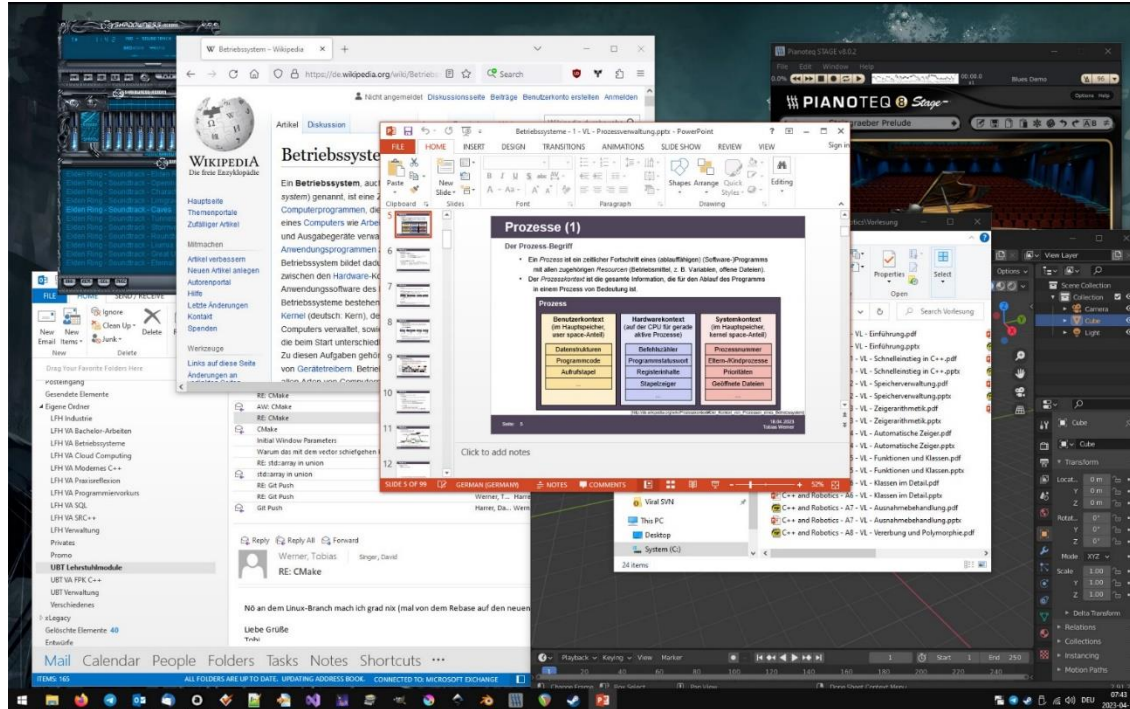
Vortragsübersicht

2 – Prozessverwaltung

- Prozesse
- Threads
- Scheduling
- Race Conditions
- Deadlocks

Prozesse (1)

Parallelisierung

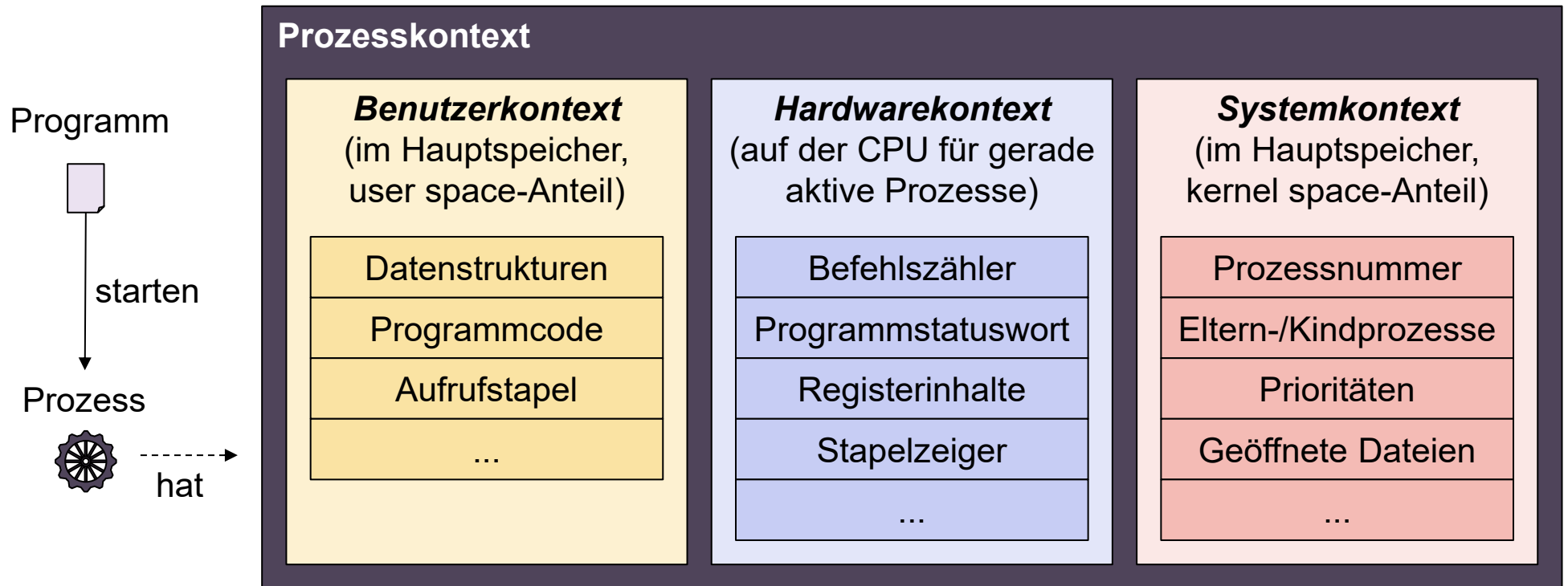


- moderne Rechner müssen mehrere Aufgaben zeitgleich abarbeiten können
(z. B. Musik abspielen, während man eine Präsentation bearbeitet)
→ *Parallelisierung, Parallelverarbeitung, Multi-Tasking* als Softwarekonzept
- mehrere *CPU-Kerne* (unabhängige Recheneinheiten) zur Leistungssteigerung

Prozesse (2)

Der Prozess-Begriff

- Ein *Prozess* ist ein zeitlicher Fortschritt eines (ablauffähigen) (Software-)Programms mit allen zugehörigen *Ressourcen* (Betriebsmittel, z. B. Variablen, offene Dateien).
- Der *Prozesskontext* ist die gesamte Information, die für den Ablauf des Programms in einem Prozess von Bedeutung ist.



[http://de.wikipedia.org/wiki/Prozesskontext#Der_Kontext_von_Prozessen_eines_Betriebssystem]

Prozesse (3)

Beispiel

C++-Code:

```
int main(int argc, char ** argv)
{
    std::cout << "wie heißt das Krokodil?";
    string krokoname; std::cin >> krokoname;

    std::cout << "Und wie viele Zähne hat es?";
    int zaehne = 0; std::cin >> zaehne;

    std::cout <<
        "Das " << krokoname <<
        "beißt Dich mit " << std::to_string(zaehne) << " Zähnen." <<
        "Du bist tot, tot und richtig tot!";
}
```



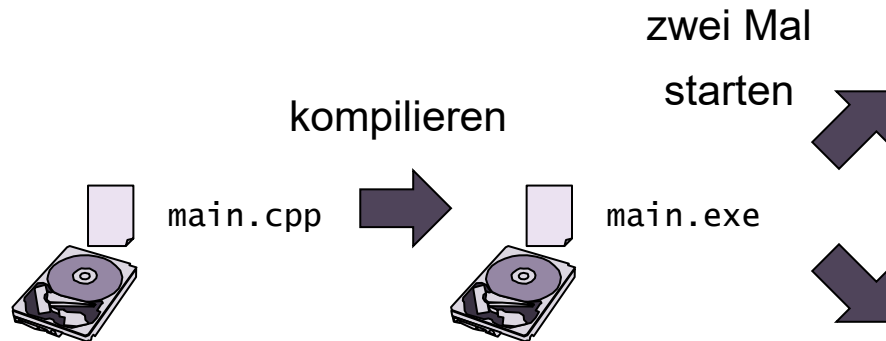
Beobachtungen:

wenn man das Programm kompiliert und zwei Mal per Doppelklick startet...

- ... erhält man zwei Kommandozeilenfenster
- ... kann man in jedem der Fenster individuell Eingaben vornehmen und weitermachen
- ... bekommt man (je nach vorheriger Eingabe) unterschiedliche Ausgaben

Prozesse (4)

Erklärung



Prozess 14



Prozesskontext (Auszug)

Aktuelle Zeile:

```
std::cout << "wie viele...
```

Variablenwerte:

```
krokoname == "Glotzi"
```

```
zaehne == ???
```

Prozess 23



Prozesskontext (Auszug)

Aktuelle Zeile:

```
std::cout << "Das " << krok...
```

Variablenwerte:

```
krokoname == "böser Bubi"
```

```
zaehne == 23
```

naive Umsetzung:

- beim Kompilieren entsteht zum Programm Quelltext ein Programm auf dem Datenträger
- das OS erzeugt für jeden Programmstart einen zugehörigen, neuen Prozess mit ID-Zahl
→ das OS teilt dem neuen Prozess einen neuen Prozesskontext zu
(d. h. gesonderten Platz im Arbeitsspeicher, etwa um sich zu merken,
welche Anweisung ausgeführt wird und welchen Wert alle Variablen haben)

Prozesse (5)

Prozesslebenszeit

Prozesse werden erzeugt durch:

- Systemstart (z. B. `init`, Daemons)
- Systemaufruf (z. B. `fork()`, `CreateProcess()`)
- Benutzeranfrage (z. B. über Terminal, Doppelklick auf Icon → Systemaufruf)
- Stapelauftrag (z. B. regelmäßige *cronjobs* → Systemaufruf)

Prozesse werden *terminiert* (beendet) durch:

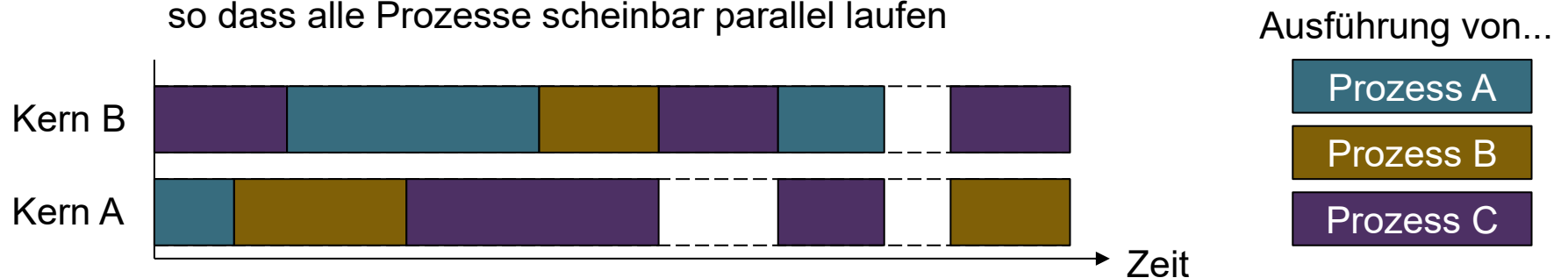
- Beendigung ihrer Arbeit (z. B. `exit()`, `exitProcess()`, Rückkehr aus `main()`)
- einfache Fehler (freiwillig → abgefangene Fehler, z. B. Datei nicht gefunden)
- schwerwiegende Fehler (unfreiwillig → nicht abgefangener Fehler, z. B. Division durch 0)
- andere Prozesse (unfreiwillig → z. B. `kill()`, `TerminateProcess()`)

[nach Tanenbaum15]

Prozesse (6)

Parallele Prozessausführung

- jeder CPU-Kern kann zu einer Zeit nur eine Maschinencode-Anweisung verarbeiten
→ *vollständiger Multiprogrammbetrieb*:
exakt n Prozesse laufen bei n CPU-Kernen (*echt parallel* (d. h. zeitgleich))
- auf typischen Systemen gibt es mehr Prozesse (z. B. > 100) als CPU-Kerne (z. B. ≤ 32)
→ *scheinbarer Multiprogrammbetrieb* bzw. *Quasi-Parallelität*:
OS wechselt pro Kern schnell (z. B. alle 16 Millisekunden)
zwischen aktuell verfügbaren Prozessen hin und her,
so dass alle Prozesse scheinbar parallel laufen



- über die Zeit flexible Zuteilung von Prozessen zu Kernen (vgl. Scheduling, später)
- zu einer gegebenen Zeit max. 1 Prozess pro Kern, max 1 Kern pro Prozess

Prozesse (7)

Konsequenzen der Parallelität

Pseudocode!

Prozess 51:

```
file = open("log.txt");  
float result =  
    /* ... Rechnung ergibt 13.04 ... */;  
file.write("Das Ergebnis der Rechnung ist ");  
file.write(result.ToString());
```

Prozess 19:

```
file = open("log.txt");  
int error =  
    /* ... Fehlercode 404 ... */;  
file.write("Der böse Fehler ist ");  
file.write(error.ToString());
```



log.txt

mögliche Datei-Inhalte bei parallelem Ablauf je nach Kontextwechseln z. B.

Das Ergebnis der Rechnung ist Der böse Fehler ist 40413.04

oder

Das Ergebnis der Rechnung ist 13.04 Der böse Fehler ist 404

das Umschalten eines CPU-Kerns von einem Prozess auf einen anderen Prozess nennt man *Kontextwechsel*, der aktuelle Prozess wird dadurch *verdrängt*

- bei typischen Betriebssystemen wirken die Kontextwechsel vermeintlich zufällig (z. B. wann das OS von welchem Prozess auf welchen anderen Prozess wechselt)
- gleichzeitig arbeitende Prozesse machen ihre Arbeit in zufälligen Stückchen, was zu Problemen führen kann (vgl. race conditions, später in dieser VL)

Prozesse (8)

Prozesszustände

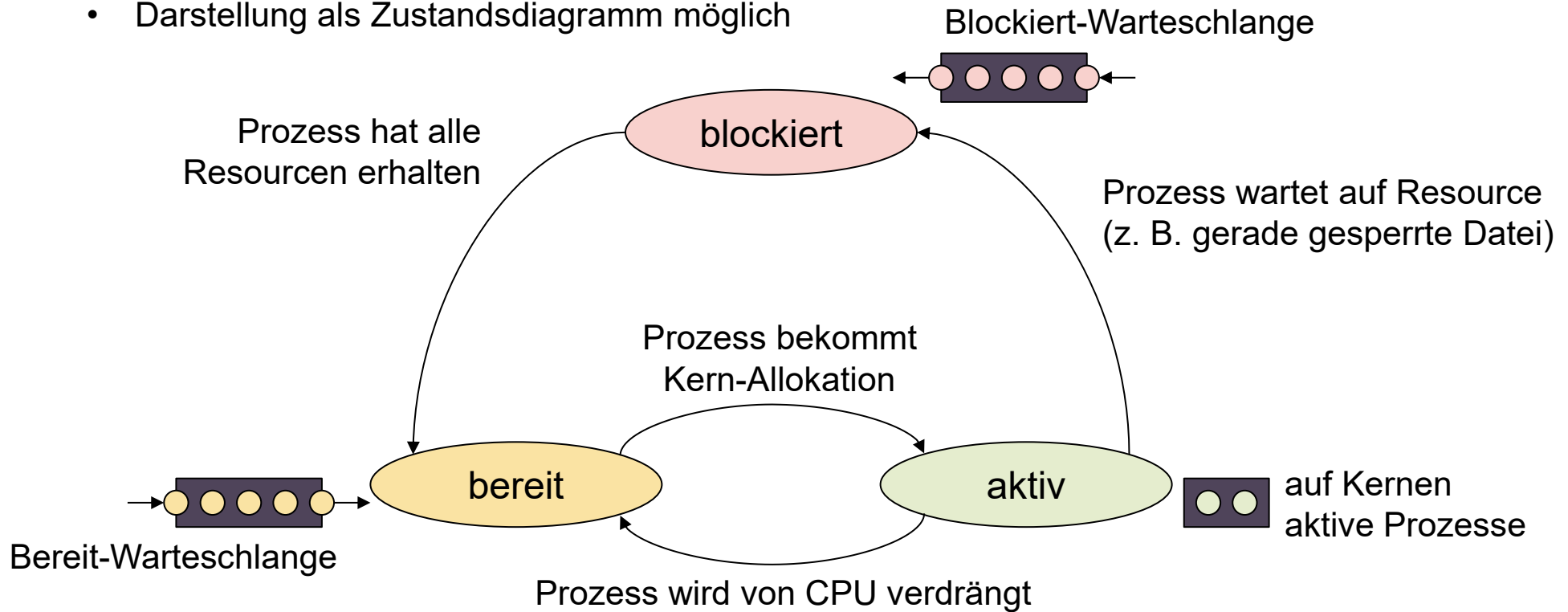
lebende Prozesse sind immer in einem von drei Zuständen:

- Prozess ist *aktiv*
 - Instruktionen des Prozesses werden gerade auf einem CPU-Kern ausgeführt
- Prozess ist *bereit*
 - Prozess könnte auf einem CPU-Kern rechnen und wartet auf Kern-Allokation
 - tritt auf, wenn es mehr Prozesse als Kerne gibt
 - bereite Prozesse warten in einer *Bereit-Warteschlange* (*ready queue*)
 - Übergang in den aktiven Zustand durch Prozesswechsel möglich
- Prozess ist *blockiert*
 - Prozess wartet auf externe Ressourcen außer einen CPU-Kern,
z. B. Ereignisse, Daten von Festplatte, Netzwerk
 - blockierte Prozesse warten in einer *Blockiert-Warteschlange* (*blocked queue*)

Prozesse (9)

Prozesszustandsdiagramm

- Prozesszustände werden vom OS verwaltet
- Darstellung als Zustandsdiagramm möglich



- individuelle Implementierungen haben erweiterte Zustandsdiagramme (z. B. andere Warteschlangen, zusätzliche Zustände)

[nach Tanenbaum15]

Vortragsübersicht

2 – Prozessverwaltung

- Prozesse
- Threads
- Scheduling
- Race Conditions
- Deadlocks

Threads (1)

Grenzen serieller Abarbeitung

in einer **einzig**en Anwendung gibt es ebenfalls oft gleichzeitig zu erledigende Aufgaben
→ serielle Abarbeitung in einem einzelnen Prozess möglich

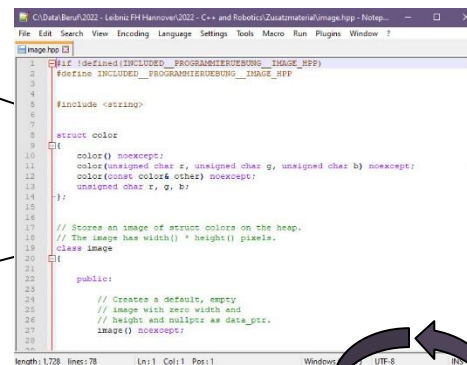
Beispiel: Anwendung zur Textverarbeitung

Tastatureingaben

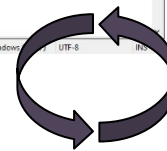
Bildschirmaktualisierung

Fensterbedienung

Nachladen von großen Dateien



```
1 #include <string>
2 #include <string>
3 #include <string>
4 #include <string>
5 #include <string>
6 #include <string>
7 #include <string>
8 #include <string>
9 #include <string>
10 #include <string>
11 #include <string>
12 #include <string>
13 #include <string>
14 #include <string>
15 #include <string>
16 #include <string>
17 #include <string>
18 #include <string>
19 #include <string>
20 #include <string>
21 #include <string>
22 #include <string>
23 #include <string>
24 #include <string>
25 #include <string>
26 #include <string>
27 #include <string>
28 #include <string>
```



zyklisches Aufrufen aller Komponenten
in einem Prozess und auf einem Kern

Nachteile:

- komplexer Programmfluss
- keine Auslastung moderner Mehrkern-CPU's
- Anwendung stockt, wenn einzelne Aufgaben untergehen

Threads (2)

Grenzen von Prozessen

in einer **einzig** Anwendung gibt es ebenfalls oft gleichzeitig zu erledigende Aufgaben
→ Auftrennen der Gesamtanwendung in mehrere einzelne Prozesse möglich

Beispiel: Anwendung zur Textverarbeitung



Nachteil:

- jeder Prozess hat einen vollständig von anderen Prozessen getrennten Prozesskontext
→ Daten zwischen Prozessen teilen ist umständlich (z. B. Dateien, Hin- und Herschicken)
→ intensive Zusammenarbeit von Prozessen schwierig

Threads (3)

Threads als Lösung

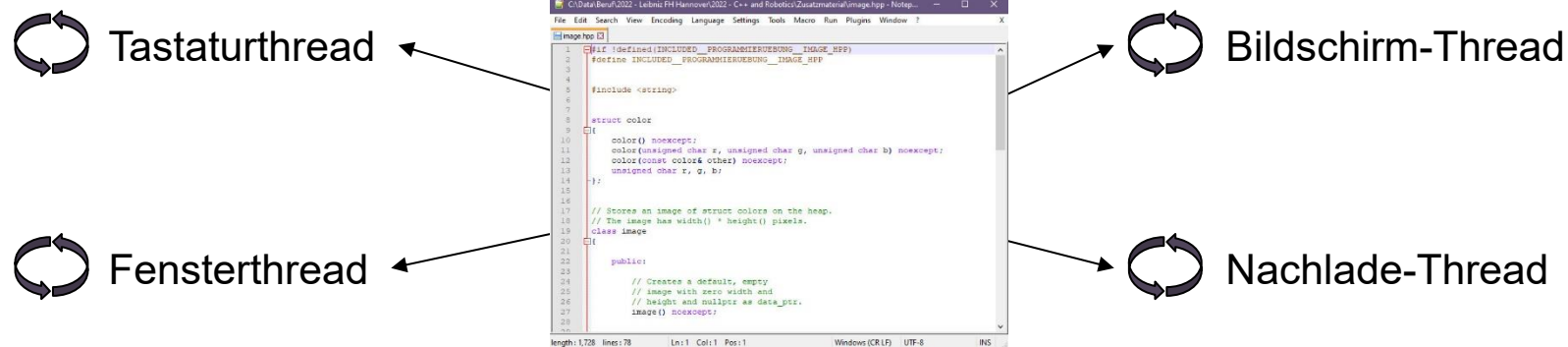
Idee *Threads* (dt. *Fäden*):

leichtgewichtige Unterprozesse, die sich viele Ressourcen (z. B. Programm, Variablen) teilen, aber einen eigenen Ausführungszustand (z. B. CPU-Kern, Stapel, aktuelle Anweisung) haben

→ *Multi-Threading*: OS kann in einem Prozess mehrere Threads ausführen

→ Prozess besteht anfangs aus einem Hauptthread, der in der `main`-Funktion startet

Beispiel: Anwendung zur Textverarbeitung



- in Threads ausgelagerte Funktionen laufen parallel (ggf. auf mehreren Kernen)
- einfache Kommunikation durch Teilen von Tastenanschlägen, Dokumententext, ...

Threads (4)

Threads und Prozesse im Vergleich

```
void function(int x)
{
    int a = x + GetInput();
    robot *r = new robot();
}
```

	zwei Prozesse	zwei Threads
geteilt	nichts	Programm vom Prozess, mit zusätzlichem Code: → alle Variablen
gesondert	Programm, aktuelle Zeile, x, a, r, Objekt zu new robot	aktuelle Zeile, ohne weiteren Code: → x, a, r, → Objekt zu new robot

Vorteile Threads:

- einfach modellierbare, parallele Abläufe innerhalb einer Anwendung
- Ausnutzung mehrerer CPU-Kerne in einer Anwendung (z. B. Auslagerung von E/A)
- leichtgewichtig: schneller zu erzeugen, zu löschen, bei "Prozess"-Wechseln
- einfacher Datenaustausch zwischen parallelen Abläufen

Nachteile Threads:

- Thread-Absturz lässt übergeordneten Prozess mitabstürzen
- Threads in einem Prozess teilen sich ungeschützt viele Daten
 - Sicherheitsrisiko durch kompromittierte Threads
 - größere Anfälligkeit für Race Conditions und Deadlocks (vgl. später)

[nach Tanenbaum15]

Vortragsübersicht

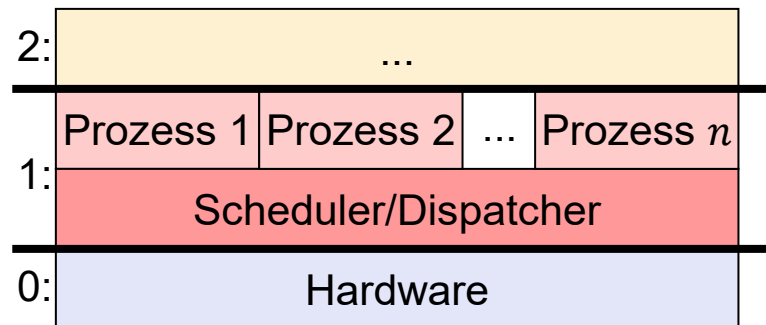
2 – Prozessverwaltung

- Prozesse
- Threads
- Scheduling
- Race Conditions
- Deadlocks

Scheduling (1)

Prozess-Scheduling in der OS-Architektur

- *Scheduler* bzw. (*Prozess-*)*Dispatcher* (Teil des OS-Kerns) regelt Zustandsübergänge
 - Verwaltung der Warteschlangen
 - Benachrichtigung bei Ressourcenverfügbarkeit
 - Durchführen von Prozesswechseln
 - *Scheduling*: Planung der Reihenfolge der CPU-Zuteilung
 - *Dispatching*: Zuteilung der CPU inklusive Prozesswechsel



Prozessverwaltung ist üblicherweise die unterste Schicht im OS

- Kontextwechsel (auch: *Prozesswechsel*, *Prozessumschaltung*, engl. *context switch*)
 - sichere Prozesskontext des verdrängten Prozesses
 - lade zuvor gesicherten Prozesskontext des neuen Prozesses

Scheduling (2)

Prozess-Scheduling als Problemstellung

gegeben:

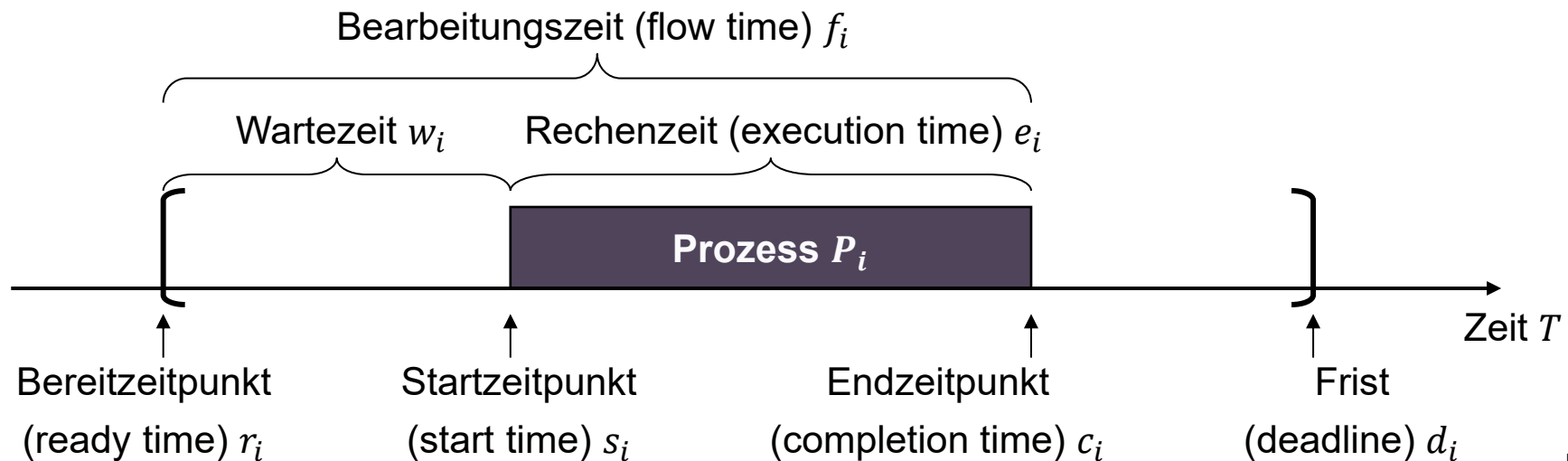
- n Prozesse (oder Threads) im Zustand bereit
- System mit einem Kern

(mehrere Kerne analog)

gesucht:

- Welcher Prozess wird auf aktiv geschaltet?

Modellierung:



[nach Baker09]

Scheduling (3)

Bewertungskriterien

Auslastung:

- Verhältnis von aufsummierter Rechenzeit $\sum_i e_i$ zu Gesamtzeit T innerhalb eines Betrachtungszeitraums, zum Beispiel $T = [0, c_{\max}]$
- CPU ist bei größeren Berechnungen das knappste Betriebsmittel des Rechners
- Ziel ist 100%-ige Auslastung der CPU, typischerweise 40% ... 90% erreichbar

Bearbeitungszeit:

- mittlere Bearbeitungszeit $f_{\text{mean}} = \frac{1}{n} \sum_i f_i$
→ enthält alle Zeiten auf der CPU und in den Warteschlangen

Gedankenspiel:

bevorzuge kurze Prozesse → kürzere mittlere Bearbeitungszeit, geringere Auslastung

bevorzuge lange Prozesse → höhere Auslastung, längere mittlere Bearbeitungszeit

→ **Es gibt keinen idealen Scheduling-Algorithmus für jede Situation!**

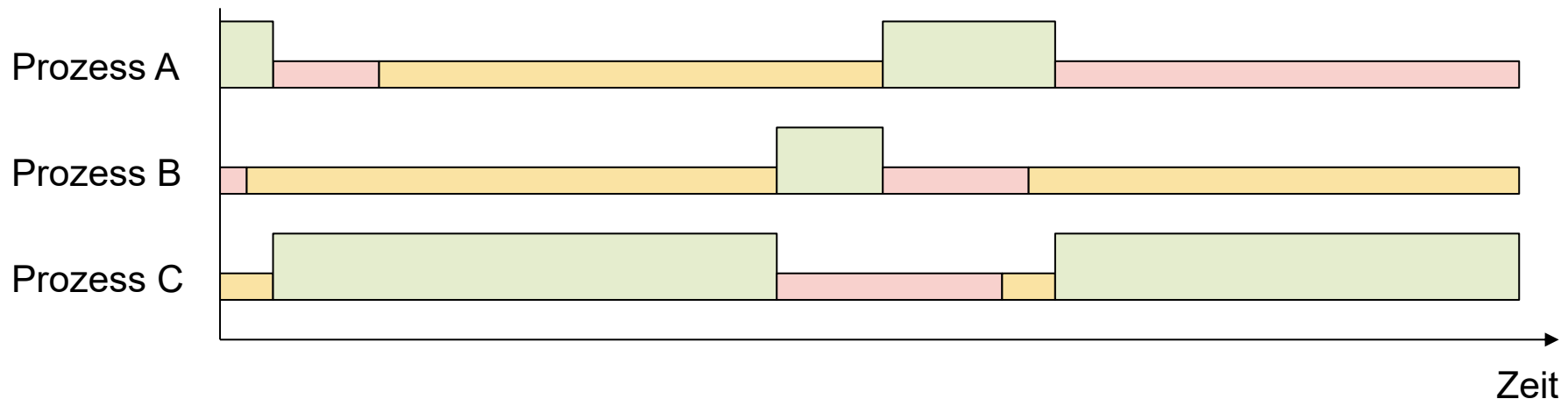
→ im Folgenden deshalb nur ein paar einfache Beispiele

Scheduling (4)

Naives Scheduling

Annahme: Prozess-(teile) müssen am Stück abgearbeitet werden (z. B. Datenbank)

Vorgehen: *First-Come-First-Serve-Scheduling* (dt. "wer zuerst kommt, mahlt zuerst"):
wähle Prozesse in der Folge der Bereitstellung



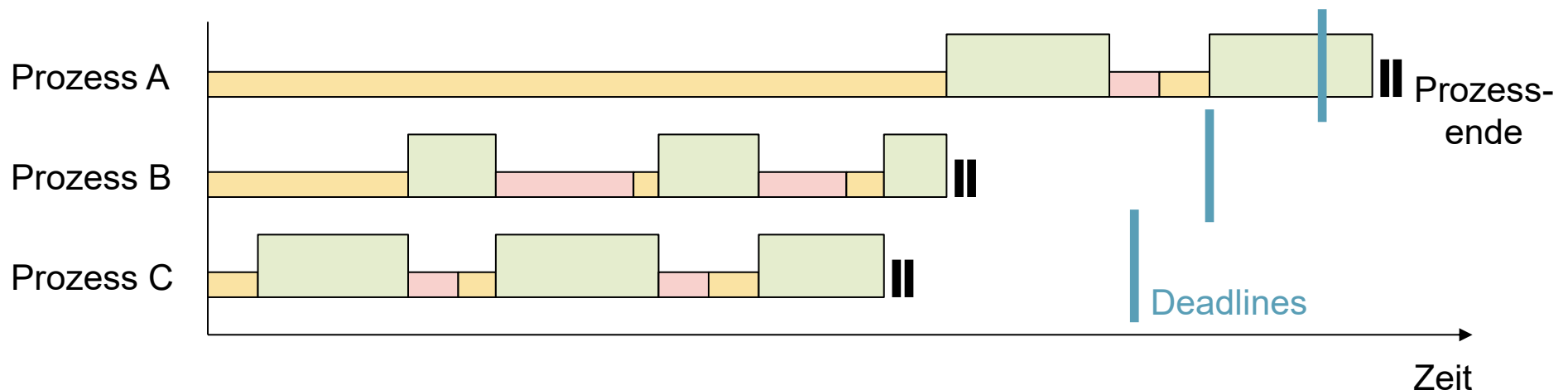
- einfache Implementierung mit FIFO-Warteschlange
- jeder Prozess kommt garantiert irgendwann an die Reihe
- ein langer, aber (minimal) früher bereiter Prozess(-teil) verzögert spätere Prozesse

Scheduling (5)

Scheduling nach Deadline

Annahme: Prozesse haben feste, kritische Deadlines (z. B. Fahrassistent)

Vorgehen: *Earliest-Deadline-First-Scheduling* (kurz *EDF-Scheduling*):
bevorzuge Prozesse, die früher fertig sein müssen



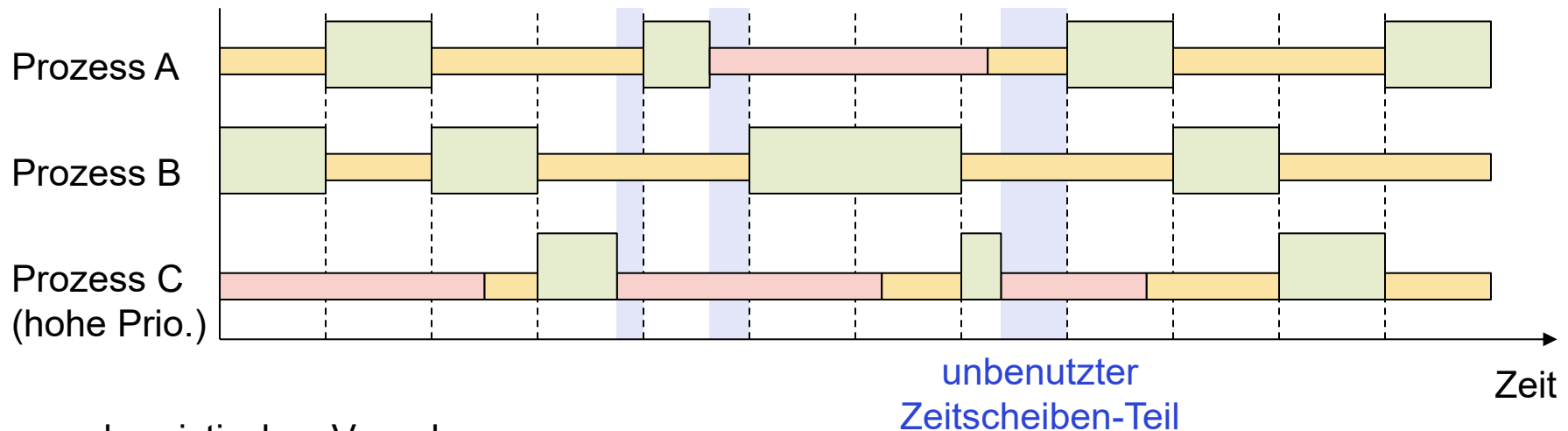
- bei wenig Last werden die Deadlines relativ sicher eingehalten
- Prozesse mit später Deadline können im Einzelfall bei hoher Auslastung verhungern, obwohl ggf. eine (alternative) Lösung zum Einhalten der Deadline existiert

Scheduling (6)

Typische Realisierung in Desktop-OS

Annahme: Prozesse dürfen beliebig verdrängt werden (*preemptive multitasking*)

Vorgehen: teile Zeit in *Zeitscheiben* (*timeslices*) und wähle für jede Zeitscheibe einen Prozess nach Bewertungskriterien (z. B. Priorität, Wartezeit)



- heuristisches Vorgehen
- guter Kompromiss aus Bearbeitungszeit und Auslastung
- Verlust von (Teil-)Zeitscheiben durch blockierende Prozesse
- im Regelfall Zusatzinfo durch den Anwender nötig
(z. B. ist ein Prozess zeitkritisch, "nur" wichtig, oder eine Hintergrund-Operation?)

Vortragsübersicht

2 – Prozessverwaltung

- Prozesse
- Threads
- Scheduling
- Race Conditions
- Deadlocks

Race Conditions (1)

Scheduling-Reihenfolge bei Threads

```
void main()
{
    std::thread thread_a(calculate_a);
    std::thread thread_b(calculate_b);

    std::cout << "waiting";
    thread_a.join();
    thread_b.join();
    std::cout << "Finished";
}
```

```
void calculate_a()
{
    for (int i = 0; i < 5; ++i)
    {
        std::cout << "a";
        std::cout << "A";
    }
}
```

```
void calculate_a()
{
    for (int i = 0; i < 5; ++i)
    {
        std::cout << "b";
        std::cout << "B";
    }
}
```

gleiches Problem wie bei Prozessen:

- die Threads laufen nach Anlegen sofort los
- ein Aufruf `t.join()` wartet, bis `t` beendet ist
- parallele Ausführung aller Threads (Hauptthread mit `main`, dazu `thread_a`, `thread_b`)
 - jeder Thread hat einen eigenen Stapel und eine eigene, aktuelle Anweisung
 - Threads laufen i. d. R. in kurzen, zufälligen Stücken auf unbestimmten CPU-Kernen
 - die Stücke sind nicht notwendigerweise ganze Programm-Anweisungen!
 - mögliche Ausgabe: `abBWaitingAabAaABbBbBaAabBAFinished`

Race Conditions (2)

Das Wettlauf-Problem

```
void main()
{
    int result = 0;
    std::thread thread_a
        (calculate_a, std::ref(result));
    std::thread thread_b =
        (calculate_b, std::ref(result));

    thread_a.join(); thread_b.join();

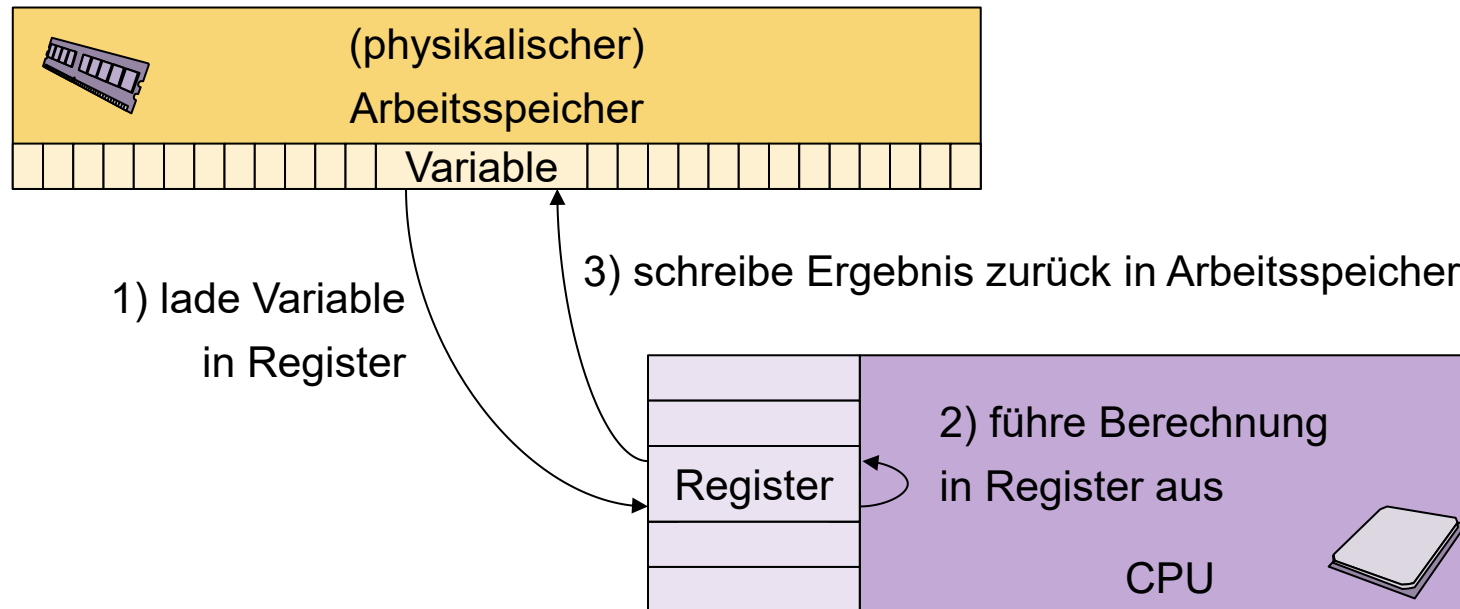
    // Ausgabe z. B.: 3, obwohl immer plus 2 gerechnet wird.
    std::cout << std::to_string(result);
}
```

```
void calculate_a(int& result)
{ ++result; ++result; }
void calculate_b(int& result)
{ ++result; ++result; }
```

- Abarbeitung in kleinen Stückchen führt zu unerwartetem Verhalten
- **Race Condition** (dt. "Wettlauf (um den Erfolg)", auch *data race*):
die Gesamtoperation ist nur dann erfolgreich, wenn jeder Thread (oder Prozess)
zufällig alle Berechnungen in einer *critical section* (einem gewissen Codeabschnitt)
am Stück macht
 - heimtückischer, schwer reproduzierbarer Fehler
 - mögliche Folgen: z. B. Programmabsturz, Datenkorruption

Race Conditions (3)

Wie rechnet eine CPU?



ein Fall von **"each abstraction leaks"** (dt. **Abstraktion ist nie blickdicht**):

- Compiler und Prozessor machen mehr, als der Programmtext explizit vorgibt
→ Transfer zu / aus CPU-Registern, Caching, Sprungvorhersagen, Optimierung, ...
- viele Umstände werden durch Programmiersprachen normalerweise verborgen...
- ... beeinflussen aber selten (z. B. bei undef. Vh. im Threading) das Programmverhalten!

Race Conditions (4)

Hintergrund für die falsche Ausgabe beim Wettlauf

```
void calculate_a(int& result) { ++result; ++result; }  
void calculate_b(int& result) { ++result; ++result; }
```

mögliche Ausführungsreihenfolge:

Thread A: lade Wert 0 aus Variable `result` in Prozessor-Register `r0`

Thread A: erhöhe Wert in Prozessor-Register `r0` um 1

Thread A: schreibe Wert 1 aus Prozessor-Register `r0` in Variable `result`

Thread B: lade Wert 1 aus Variable `result` in Prozessor-Register `r1`

Thread A: lade Wert 1 aus Variable `result` in Prozessor-Register `r0`

Thread B: erhöhe Wert in Prozessor-Register `r1` um 1

Thread B: schreibe Wert 2 aus Prozessor-Register `r1` in Variable `result`

Thread A: erhöhe Wert in Prozessor-Register `r0` um 1

Thread A: schreibe Wert 2 aus Prozessor-Register `r0` in Variable `result`

Thread B: lade Wert 2 aus Variable `result` in Prozessor-Register `r1`

Thread B: erhöhe Wert in Prozessor-Register `r1` um 1

Thread B: schreibe Wert 3 aus Prozessor-Register `r1` in Variable `result`

← überschreibt
Ergebnis von
Thread B

→ kleine Stückchen können auch Teile einer einzelnen Rechenoperation sein!

→ bei Prozessen seltener Fehler als bei Threads, weil (fast) keine geteilten Variablen

Race Conditions (5)

Race Conditions in der Praxis

```
void main()
{
    std::string result;
    std::thread thread_a
        (calculate_a, std::ref(result));
    std::thread thread_b
        (calculate_b, std::ref(result));

    thread_a.join();
    thread_b.join();
    std::cout << result; // undef. vh.
}
```

```
void calculate_a(std::string& r)
{ r = "Alice"; } // undef. vh.
void calculate_b(std::string& r)
{ r = "Bob"; } // undef. vh.

Thread A: free(r.data_)
Thread B: free(r.data_) // !!
Thread A: r.data_ = malloc(5)
Thread B: r.data_ = malloc(3) // !!
Thread A: memcpy(r.data_, "Alice") // !!
Thread B: memcpy(r.data_, "Bob")
Thread B: r.size_ = 3
Thread A: r.size_ = 5 // !!
```

je nach Sprache Speicherlecks und Abstürze durch Race Conditions, z. B.:

- bei Objekten mit Invarianten kann Threading die Invarianten kaputtschießen
- bei Speicherverwaltung kann Threading die `malloc` / `free`-Reihenfolge stören

Variablen (ggf. ganze Objekte, Datenstrukturen, Dateien, ...) dürfen in der Regel zur gleichen Zeit nur von einem Thread benutzt werden.

→ Wie stellt man das sicher, wenn man Daten zwischen Threads teilen will?

Race Conditions (6)

Mutexing zur Vermeidung von Race Conditions

Pseudo-Code!

```
mutex m;

void calculate_a(std::string& r)
{
    m.lock();           // warte bis Mutex frei ist, dann belege Mutex
    r = "Alice";
    m.unlock();         // gebe Mutex frei (und z. B.: schreibe Register in Speicher zurück)
}

void calculate_b(std::string& r)
{
    m.lock();           // warte bis Mutex frei ist, dann belege Mutex
    r = "Bob";
    m.unlock();         // gebe Mutex frei (und z. B.: schreibe Register in Speicher zurück)
}
```

Lösung: *Mutex* (kurz für "*Mutual Exclusion*" dt. Belegungsflagge) als Konzept

- jeder Mutex kann nur von einem Thread gleichzeitig belegt werden
- andere Threads warten ggf. beim Belegen, bis der Mutex wieder freigegeben ist
(Varianten: *idle waiting* oder Wechsel auf Blockiert-Prozesszustand)
- Belegen und Freigeben von Mutexen löst mit Systemaufruf technische Details
(z. B. Register-Synchronisation mit Caches und dem Hauptspeicher)

Race Conditions (7)

Mutexing in der Praxis

Pseudo-Code!

```
void main()
{
    std::string result;
    mutex m;
    std::thread thread_a
        (calculate_a,
         std::ref(result), std::ref(m));
    thread thread_b =
        (calculate_b,
         std::ref(result), std::ref(m));

    thread_a.join();
    thread_b.join();

    // Ausgabe entweder "Alice" oder "Bob"
    std::cout << result;
}
```

```
void calculate_a
    (std::string& r, mutex& m)
{
    m.lock();    // belege Mutex
    r = "Alice";
    m.unlock();  // gebe Mutex frei
}

void calculate_b
    (std::string& r, mutex& m)
{
    m.lock();    // belege Mutex
    r = "Bob";
    m.unlock();  // gebe Mutex frei
}
```

- erzwingt sequentielle Abarbeitung von kritischen Operationen mit einem Mutex
→ kein undefiniertes Verhalten durch Race Conditions mehr
- Abarbeitungsreihenfolge bis zum Mutex weiterhin undefiniert
→ es gewinnt hier im Beispiel der Thread, der den Mutex als zweites bekommt

Race Conditions (8)

Threadsicherheit durch Mutexing

Pseudo-Code!

```
class AtomicInt
{
    private: mutex m_;
    private: int value_;
    public: void set(int value) // threadsafe
    {
        m_.lock();
        value_ = value;
        m_.unlock();
    }
    public: int get() // threadsafe
    {
        // Rückgabe ist hier per Kopie nötig, statt per ref!
        m_.lock(); int result = value_; m_.unlock(); return result;
    }
};
```

```
// Thread A
void calculate_a(AtomicInt& i)
{ i.set(17); }

// Thread B
void calculate_b(AtomicInt& i)
{
    std::cout <<
        std::to_string(i.get());
}
```

- Mutexe können als Membervariablen auftreten und Membermethoden sichern
- eine Methode ist *threadsicher*: sie darf von beliebigen Threads aufgerufen werden
→ manchmal auch: die Methode ist *TS-atomar* (das ist nicht AS- oder DB-atomar)
- eine Klasse ist *threadsicher*: alle mehrmals aufrufbaren Methoden sind threadsicher
(also alle Methoden außer Konstruktoren, Destruktoren)

Vortragsübersicht

2 – Prozessverwaltung

- Prozesse
- Threads
- Scheduling
- Race Conditions
- Deadlocks

Deadlocks (1)

Wechselweises Warten auf Mutexfreigabe

Pseudo-Code!

```
string vorname;      mutex m_vorname;  
string nachname;     mutex m_nachname;
```

```
void calculate_a()    // Läuft in Thread A  
{  
    m_vorname.lock();    (1)  
    vorname = "Alice";  
  
    m_nachname.lock();   (3)  
    nachname = "Liddell";  
  
    m_nachname.unlock();  
    m_vorname.unlock();  
}
```

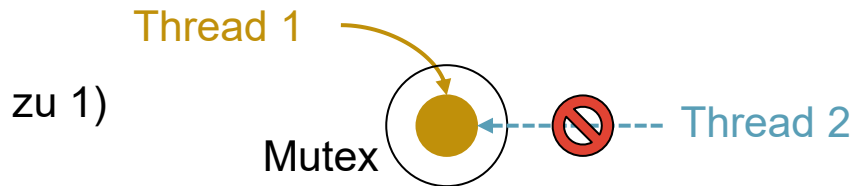
```
void calculate_b()    // Läuft in Thread B  
{  
    m_nachname.lock();   (2)  
    nachname = "Schwammkopf";  
  
    m_vorname.lock();  
    vorname = "Bob";  
  
    m_vorname.unlock();  
    m_nachname.unlock();  
}
```

- **Deadlock:** Zwei Threads warten unbegrenzt lange auf die Freigabe eines Mutex, den der jeweils andere Thread belegt hält. Da beide Threads warten, kann keiner der Threads für den jeweils anderen Thread den Mutex freigeben.
 - Symptom: Programm bleibt manchmal ("zufällig") beim Ausführen hängen
 - Grund: Problem tritt oft nur bei falscher ("zufälliger") Abarbeitungsreihenfolge auf
(Hier: Thread B führt (2) zwischen (1) und (3) von Thread A aus)

Deadlocks (2)

Voraussetzungen für Deadlocks

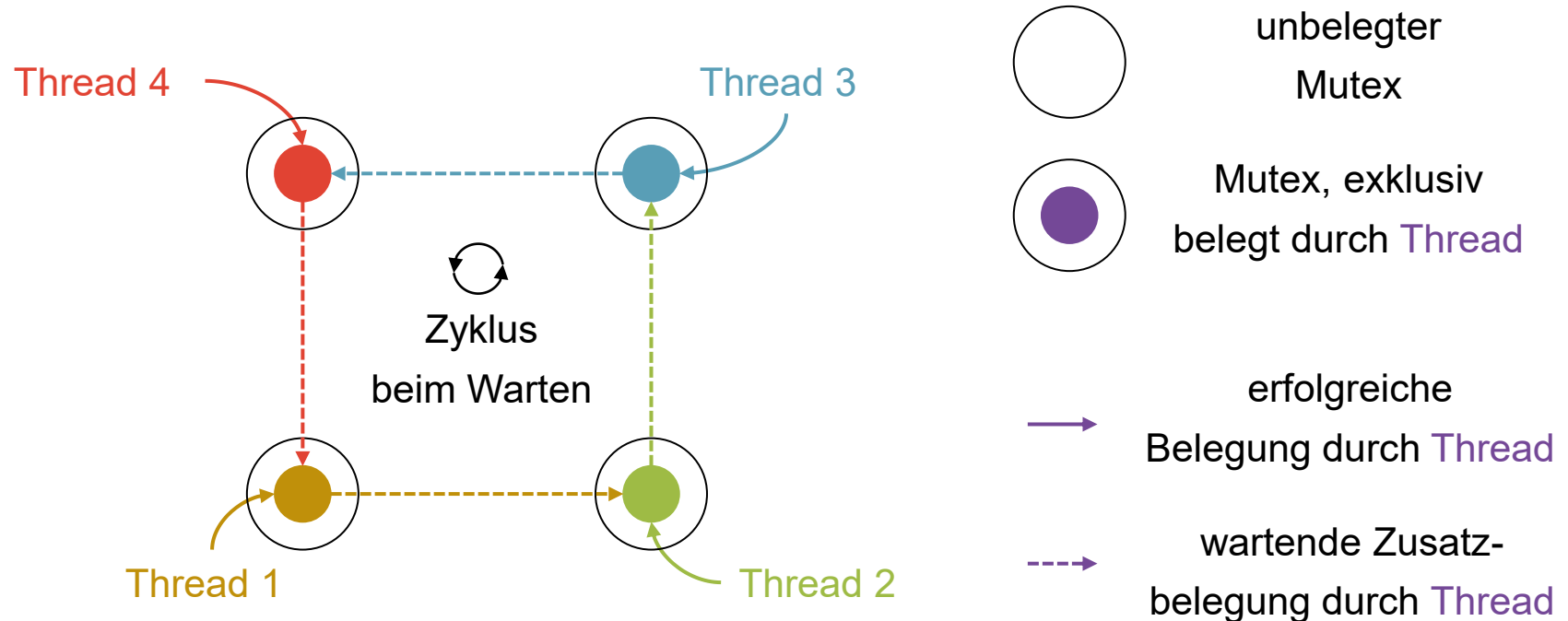
- 1) Mutexe sind *exklusiv* (engl. *mutual exclusion*)
 - ein Thread kann einen anderweitig belegten Mutex nicht direkt belegen
- 2) *keine externe Mutexfreigabe* (engl. *no preemption*)
 - nur der belegende Thread kann belegte Mutexe **ausdrücklich** freigeben
 - Gegenbeispiel: der belegende Thread verliert den Mutex nach einer gewissen Zeit
- 3) *Zusatzbelegung von Mutexen* (engl. *hold and wait*)
 - ein Thread wartet auf einen belegten Mutex, nachdem er einen Mutex belegt hat
- 4) *Zyklus in Mutex-Belegungen* (engl. *circular wait*)
 - es kann sich ein Kreis aufeinander wartender Threads einstellen



Deadlocks (3)

Umgang mit Deadlocks

- Darstellung der Deadlock-Situation am *Betriebsmittelgraph*



- die vier Bedingungen sind *hinreichend* und *notwendig*
 - hinreichend: wenn alle vier Bedingungen vorliegen, gibt es immer ein Deadlock
 - notwendig: wenn ein Deadlock auftritt, sind immer alle vier Bedingungen erfüllt
 - um Deadlocks zu vermeiden, genügt es, eine Bedingung zu vermeiden

Deadlocks (4)

Verhindern von Deadlocks

1) Mutexe sind exklusiv

→ das liegt an der Natur der Dinge, hier haben wir kaum eine Wahl

2) keine externe Mutexfreigabe

→ Deadlock-Erkennung: Zwingt auf Mutex wartende Threads nach Wartedauer, alle bislang belegten Mutexe freizugeben und später neu zu probieren

→ Hoffnung: Zufällig entsteht irgendwann eine passende Abarbeitungsreihenfolge

→ unpraktische Umsetzung durch komplizierten Programmfluss

3) Zusatzbelegung von Mutexen

→ nur einen Mutex für alle Objekte

→ dann kann ich mir Threads auch sparen...

→ alle nötigen Mutexe gleichzeitig belegen

→ wird OS-seitig oft nicht richtig unterstützt

4) Zyklus in Mutex-Belegungen

→ Ressourcen immer in der selben Reihenfolge belegen

Deadlocks (5)

Deadlock-Auflösung in der Praxis

```
string vorname;      mutex m_vorname;  
string nachname;     mutex m_nachname;
```

```
void calculate_a()    // läuft in Thread A  
{  
    m_vorname.lock();  
    vorname = "Alice";  
  
    m_nachname.lock();  
    nachname = "Liddell";  
  
    m_nachname.unlock();  
    m_vorname.unlock();  
}
```

```
void calculate_b()    // läuft in Thread B  
{  
    m_vorname.lock();  
    vorname = "Bob";  
  
    m_nachname.lock();  
    nachname = "Schwammkopf";  
  
    m_nachname.unlock();  
    m_vorname.unlock();  
}
```

- Reihenfolge der Mutexbelegung ist jetzt immer gleich
- unabhängig von Abarbeitungsreihenfolge kein Deadlock mehr möglich
- Mutexe auch in umgekehrter Reihenfolge freigeben!

Deadlocks (6)

Race Conditions trotz Mutexing

```
mutex m;  
string vorname;  
string nachname;
```

```
void calculate_a()    // läuft in Thread A  
{  
    m.lock();  
    vorname = "Alice";  
    m.unlock();  
  
    m.lock();  
    nachname = "Liddell";  
    m.unlock();  
}
```

```
void calculate_b()    // läuft in Thread B  
{  
    m.lock();  
    vorname = "Bob";  
    m.unlock();  
  
    m.lock();  
    nachname = "Schwammkopf";  
    m.unlock();  
}
```

```
// mögliche Ergebnisse:  
// Alice Liddell, Bob Liddell, Alice Schwammkopf, Bob Schwammkopf
```

- je nachdem, welche Mutexe man wann setzt, kommen unterschiedliche Reihenfolgen und Ergebnisse in Frage
→ ob etwas tatsächlich eine Race Condition ist, hängt von der Anwendung ab

Zusammenfassung

Prozesse und Threads

- Prozesse entsprechen laufenden Instanzen von Programmen
- Abarbeitung in einzelnen Stückchen auf ein oder mehreren CPU-Kernen
- Threads sind leichte (Unter-)Prozesse eines Programms mit geteiltem Adressraum
- es gibt kein optimales Scheduling, aber anwendungs-spezifische Lösungen

Race Conditions und Deadlocks

- unregelmäßiger, paralleler Zugriff auf Betriebsmittel (z. B. Variablen) führt zu Problemen
→ undefiniertes Verhalten, Datenkorruption, Programmabsturz, ...
- Mutex kann nur von einem Thread belegt werden, andere Threads müssen warten
→ Synchronisation gemeinsam genutzter Daten
- Deadlock ist Folge wechselseitigen Wartens auf Ressourcenfreigabe (z. B. Mutex)
→ allgemeine Lösung schwierig, z. B. Mutexe in fester Reihenfolge belegen